

Structural Budget Model: Data, Features, Formulas, Training and Testing

Process overview (part-by-part)

- Part 1: Data acquisition and schema normalization (CSV import, header cleaning).
- Part 2: Data cleaning and target transformation (NaN handling, $\log(1 + \cdot)$).
- Part 3: Feature engineering (regex parsing, derived totals/ratios).
- Part 4: Feature matrix construction (9 raw engineered inputs).
- Part 5: Polynomial expansion (degree 2) and standardization (z-score).
- Part 6: Train/test split (80/20 holdout).
- Part 7: Baseline model training (LSBoost) and evaluation.
- Part 8: Feature selection via ensemble importances (median threshold).
- Part 9: Hyperparameter optimization (Bayesian) and final model training.
- Part 10: Final evaluation, artifacts saving, and inference pipeline.

Variables glossary (concise definitions)

Clean name	Source header	Type	Unit	Definition / Role
project	Project	text	n/a	Project title; used to parse <code>num_storeys</code> , <code>num_classrooms</code> via regex.
year	Year	numeric	year	Project year; raw feature.
budget	Budget	numeric	PHP	Target variable; filtered to >100,000 then log-transformed.
num_storeys	parsed from project	numeric	count	Integer before “STY”; median-imputed if missing.
num_classrooms	parsed from project	numeric	count	Integer before “CL”; median-imputed if missing.
Derived structural aggregates				
total_gross_floor_area	sum of columns containing “grossfloorarea”	numeric	sq.m.	Total gross floor area across all floor-finish variants.
total_concrete_volume	sum of columns containing “volumeofstructuralconcrete”	numeric	cu.m.	Total structural concrete volume across categories.
total_reinforcing_steel	Grade 40 + Grade 60	numeric	kg (assumed)	Sum of reinforcing steel by grade.
areaofformworks_sq_m__	any column containing “formworks”	numeric	sq.m.	Area of formworks; auto-detected; used as-is.
Normalized ratios				
concrete_per_floor_area	derived	numeric	cu.m./sq.m.	<code>total_concrete_volume / total_gross_floor_area</code> ; NaN/Inf -> 0.
steel_per_concrete	derived	numeric	ratio	<code>total_reinforcing_steel / total_concrete_volume</code> ; NaN/Inf -> 0.

Exact cleaned variable names (after `makeValidName` + lower)

Examples from CSV to cleaned names:

- Budget → `budget`; Year → `year`; Grade 40 → `grade40`; Grade 60 → `grade60`.
- Gross floor area (Cement floor finish) → `grossfloorarea_cementfloorfinish` (pattern: contains “grossfloorarea”).

- Volume of structural concrete (cu.m.) ... → `volumeofstructuralconcrete_cu_m_...` (pattern: contains “volumeofstructuralconcrete”).
- Area of formworks (sq.m.) → `areaofformworks_sq_m_` (auto-detected by substring “formworks”).

Final raw feature set (9 inputs) and counting proof

The structural training script fixes the raw input vector to the following 9 features (see Part 4 in `main.m`):

$X_{\text{raw}} = [\text{year}, \text{num_storeys}, \text{num_classrooms}, \text{total_gross_floor_area}, \text{total_concrete_volume}, \text{total_reinforcing_steel}, \text{areaofformworks_sq_m_}, \text{concrete_per_floor_area}, \text{steel_per_concrete}]$

Count: $p = 9$. Degree-2 polynomial design yields exactly

$$\underbrace{p}_{\text{originals}} + \underbrace{p}_{\text{squares}} + \underbrace{\binom{p}{2}}_{\text{pairwise interactions}} = 9 + 9 + 36 = 54 \text{ features.}$$

Why exactly 9 input features? Justification of Feature Engineering

The structural training code fixes the raw input vector to 9 features to avoid target leakage while capturing the project’s structural scope and material intensity. This curated set provides a robust and interpretable basis for the model.

$$\underbrace{1}_{\text{year}} + \underbrace{2}_{\text{parsed from project}} + \underbrace{4}_{\text{structural aggregates}} + \underbrace{2}_{\text{normalized ratios}} = 9.$$

- **Year (1):** `year` is included to account for temporal variations, such as inflation in material and labor costs over time.
- **Parsed from Project (2):** `num\_storeys` and `num\_classrooms` are extracted from the project title via regular expressions. They serve as high-level proxies for the building’s vertical scale and programmatic size, which are fundamental drivers of structural cost.
- **Structural Aggregates (4):** `total\_gross\_floor\_area`, `total\_concrete\_volume`, `total\_reinforcing\_steel`, and `areaofformworks\_sq\_m_` represent the core material quantities that define the structural system. They are aggregated from more granular data to create stable, summary predictors of the primary structural components.
- **Normalized Ratios (2):** `concrete\_per\_floor\_area` and `steel\_per\_concrete` are engineered to measure material intensity. By normalizing structural mass (concrete, steel) relative to the building’s size or other materials, these ratios capture the structural system’s efficiency and complexity, allowing the model to generalize better across projects of different scales. For instance, a higher `steel\_per\_concrete` ratio might indicate a more complex or load-intensive design.

Complete polynomial feature list (54 features)

Given $p = 9$ inputs, `generatePolyFeatures(T,2)` emits:

Original 9

year, num_storeys, num_classrooms, total_gross_floor_area, total_concrete_volume, total_reinforcing_steel, areaofformworks_sq_m__, concrete_per_floor_area, steel_per_concrete

9 squares

year^2, num_storeys^2, num_classrooms^2, total_gross_floor_area^2, total_concrete_volume^2, total_reinforcing_steel^2, areaofformworks_sq_m__^2, concrete_per_floor_area^2, steel_per_concrete^2

36 pairwise interactions

- year*num_storeys
- year*num_classrooms
- year*total_gross_floor_area
- year*total_concrete_volume
- year*total_reinforcing_steel
- year*areaofformworks_sq_m__
- year*concrete_per_floor_area
- year*steel_per_concrete
- num_storeys*num_classrooms
- num_storeys*total_gross_floor_area
- num_storeys*total_concrete_volume
- num_storeys*total_reinforcing_steel
- num_storeys*areaofformworks_sq_m__
- num_storeys*concrete_per_floor_area
- num_storeys*steel_per_concrete
- num_classrooms*total_gross_floor_area
- num_classrooms*total_concrete_volume
- num_classrooms*total_reinforcing_steel
- num_classrooms*areaofformworks_sq_m__
- num_classrooms*concrete_per_floor_area
- num_classrooms*steel_per_concrete
- total_gross_floor_area*total_concrete_volume
- total_gross_floor_area*total_reinforcing_steel
- total_gross_floor_area*areaofformworks_sq_m__
- total_gross_floor_area*concrete_per_floor_area
- total_gross_floor_area*steel_per_concrete
- total_concrete_volume*total_reinforcing_steel
- total_concrete_volume*areaofformworks_sq_m__
- total_concrete_volume*concrete_per_floor_area
- total_concrete_volume*steel_per_concrete
- total_reinforcing_steel*areaofformworks_sq_m__
- total_reinforcing_steel*concrete_per_floor_area
- total_reinforcing_steel*steel_per_concrete
- areaofformworks_sq_m__*concrete_per_floor_area
- areaofformworks_sq_m__*steel_per_concrete
- concrete_per_floor_area*steel_per_concrete

Polynomial/interaction expansion and scaling

We build degree-2 features via `generatePolyFeatures(T,2)` and compute z-scores on the training split only:

$$\mu_j = \frac{1}{n} \sum_i X_{ij}, \quad \sigma_j = \sqrt{\frac{1}{n-1} \sum_i (X_{ij} - \mu_j)^2}, \quad Z_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j}, \quad \text{NaN} \rightarrow 0.$$

Train/test split and targets

We hold out 20% using `cvpartition(HoldOut=0.2)`. The response is $y = \log(1 + \text{budget})$; predictions are inverted with $\exp(\cdot) - 1$ for reporting in PHP.

Baseline and optimized models

Baseline uses LSBoost with `NumLearningCycles=100` on the scaled 54 features.

- Metrics on the held-out test set: report R^2 and MAE; see figures below.

Feature selection keeps features with importance above the median (per `predictorImportance`), reducing variance and focusing on predictive interactions. **Optimized model** is LSBoost with hyperparameters tuned via Bayesian optimization over: cycles [500,1000], splits [8,32], learn rate [0.01,0.1] (log-scale).

Figures (testing evidence)

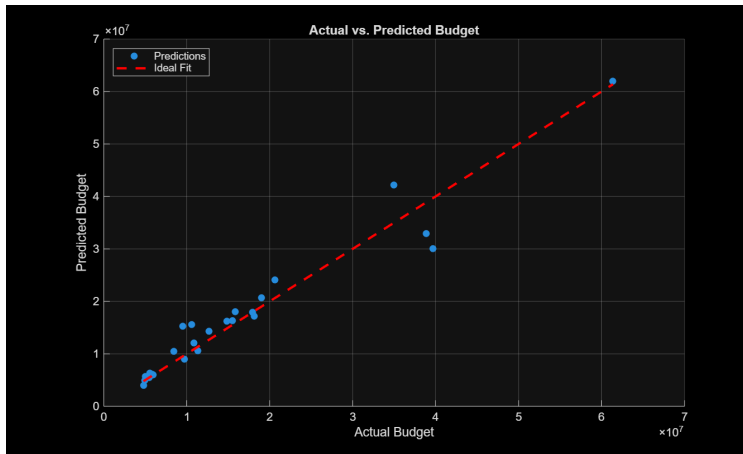


Figure 1: Actual vs. Predicted (held-out test). Tight clustering along the diagonal indicates good fit.

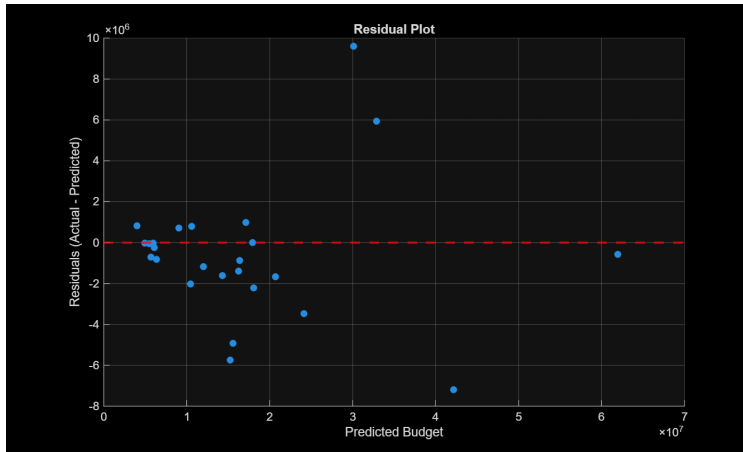


Figure 2: Residuals vs. Predicted. Random scatter around zero suggests low bias and well-calibrated variance.

Testing metrics (numerical summary)

To document exact test-set performance (R^2 , MAE, selected feature count), we include an auto-generated snippet if available.

(Run `export_structural_metrics.m` to generate and include the exact metrics with a fixed RNG seed.)

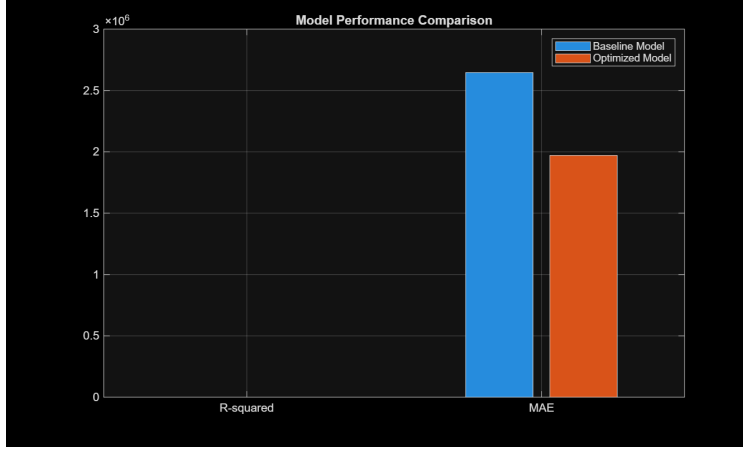


Figure 3: Baseline vs. Optimized performance (R^2 , MAE) on test split.

Testing metrics: definitions

Given actual budgets in PHP on the test split (y_i) and predictions ($\hat{y}_i$) after inversion from log-space:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad \text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|.$$

The exported snippet (if present) reports these exact values for both baseline and optimized models.

Quantitative justification of testing

- Metrics are computed on the 20% holdout only; no training rows are reused.
- Predictions are produced in log-space and inverted with $\exp(\cdot) - 1$ before computing MAE and R^2 in PHP.
- Feature selection threshold is the median of baseline importances; the included metrics snippet reports the exact count retained.

Exact numbers behind feature-importance

For tree ensembles like the LSBoost model used here, feature importance is calculated by summing the reductions in Mean Squared Error (MSE) at every split in every tree where that feature is used. The reduction is weighted by the number of observations at the node being split. This method, often called "Gini importance" or "mean decrease in impurity," quantifies how much a feature contributes to improving the model's accuracy.

The formula for the importance of a single feature j is:

$$\text{Imp}(j) = \sum_{t \in \text{trees}} \sum_{n \in \mathcal{N}_{t,j}} (\Delta \text{MSE}(n) \times w(n))$$

where:

- $\mathcal{N}_{t,j}$ is the set of nodes in tree t that use feature j for a split.
- $\Delta \text{MSE}(n)$ is the reduction in MSE achieved by splitting node n .

- $w(n)$ is a weight representing the number of training samples at node n .

The bar chart below visualizes these raw importance scores for the top-20 features, sorted in descending order.

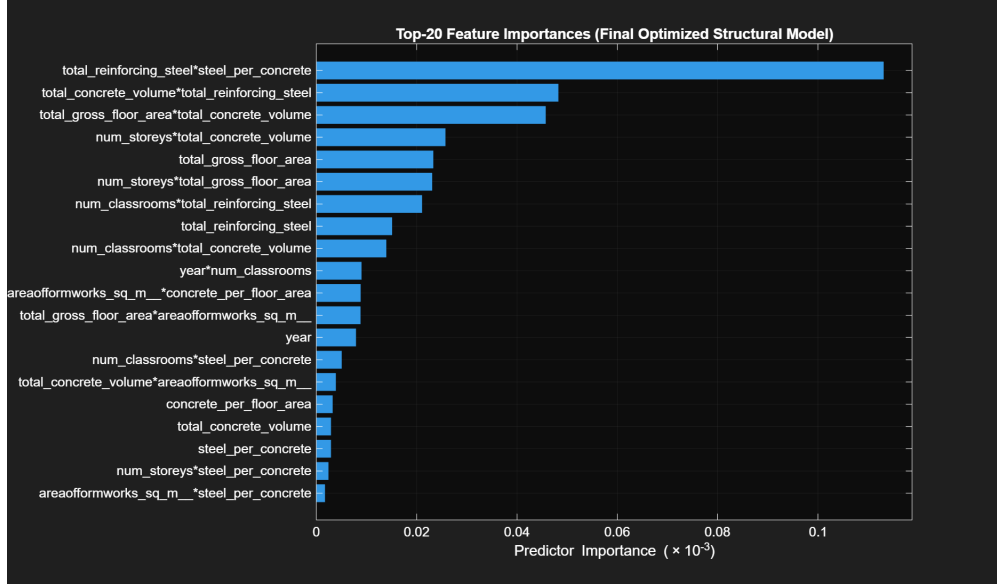


Figure 4: Top-20 feature importances of the optimized structural ensemble.

If generated, the exact table will be included below: (Run `extract_feature_importances_structural.m` to export and include exact values.)

Quantitative justification (using extracted values)

Using the optimized structural ensemble's exact importances (Table ??):

- Top feature: `total_reinforcing_steel*steel_per_concrete` = 29.10% of total importance.
- Next: `total_concrete_volume*total_reinforcing_steel` = 12.41%; `total_gross_floor_area*total_concrete_volume` = 11.76%.
- Additional leaders: `num_storeys*total_concrete_volume` = 6.62%; `total_gross_floor_area` = 6.00%; `num_storeys*total_gross_floor_area` = 5.94%.
- Cumulative contributions: top-3 = 53.27%, top-5 = 65.89%, top-10 = 87.05% of total importance (rounding may cause small discrepancies).

Interpretation:

- **Steel intensity and mass dominate:** The leading term `steel*steel_per_concrete` (29.10%) and `concrete*steel` (12.41%) show that reinforcement quantity together with reinforcement intensity per concrete volume is the primary driver.
- **Concrete and scale effects:** `floor_area*concrete_volume` (11.76%) and `num_storeys*concrete_volume` (6.62%) indicate joint influence of building size and structural mass on cost.
- **Gross floor area is independently predictive:** `total_gross_floor_area` contributes 6.00% on its own, with `num_storeys*total_gross_floor_area` adding 5.94%.
- **Program size and time provide secondary signal:** `num_classrooms*total_reinforcing_steel` (5.42%), `year*num_classrooms` (2.32%), and `year` (2.04%).

- **Formworks and intensities add refinements:** `areaofformworks*concrete_per_floor_area` (2.27%), `floor_area*areaofformworks` (2.27%), `concrete_volume*areaofformworks` (1.00%), with base intensities `concrete_per_floor_area` (0.84%) and `steel_per_concrete` (0.75%).

Model Architecture: Why No Hidden Layers?

The model used for structural budget prediction is a **Least-Squares Boosting (LSBoost) ensemble**, a powerful machine learning technique based on decision trees. It is fundamentally different from a neural network and, as a result, **does not have hidden layers**. This section explains the model's structure, its parameters, and the importance of this architectural choice.

Architectural Parameters (The Analog to "Layers")

Instead of layers, weights, and activation functions, the complexity of an LSBoost model is defined by its component trees. The key architectural parameters are:

- **Number of Trees (NumLearningCycles):** This is the total number of decision trees in the ensemble. Each tree is trained sequentially to correct the errors of the one before it. This is the most direct analog to the "depth" or "size" of a neural network.
 - **Baseline Model:** Uses a fixed number of **100 trees**.
 - **Optimized Model:** The number of trees is automatically tuned by searching for the best value within the range of **500 to 1000 trees**.
- **Tree Depth (MaxNumSplits):** This parameter controls the complexity of each individual tree by limiting how many decision points (splits) it can have.
 - **Baseline Model:** Uses the default value of **10 splits**.
 - **Optimized Model:** The depth is tuned by searching for the best value between **8 and 32 splits**.

Justification for Hyperparameter Ranges in the Optimized Model

The optimized model does not have a single, fixed number for its parameters because it uses **Bayesian optimization**. This is an intelligent and automated process for finding the best-performing combination of hyperparameters.

- **Search Space:** We define a *range* of possible values for each parameter (e.g., '[500, 1000]' for trees).
- **Automated Search:** The algorithm iteratively trains different model versions, using the results from past attempts to intelligently decide which combination to try next. This is far more efficient than manual trial-and-error.
- **Result:** The final "optimized" model is the one that performed best during this search. Its exact number of trees and splits are an *output* of the training process itself, not a predetermined input. The ranges specified here represent the boundaries of that search.

Why This Architecture is Important

Choosing a tree-based ensemble like LSBoost over a neural network offers several key advantages for this specific problem:

- **Interpretability:** Unlike the "black box" nature of neural networks, tree models provide clear **feature importances**. As shown in Figure ??, we can see exactly which inputs (e.g., the interaction between steel quantity and steel intensity) are the most influential drivers of the budget. This is crucial for understanding the cost structure and building trust in the model's predictions.
- **Performance on Tabular Data:** For structured, tabular data like this project's dataset, tree-based ensembles often outperform neural networks. They are highly effective at capturing complex, non-linear interactions between features.
- **Robustness:** The boosting process, where each tree corrects the errors of its predecessors, makes the model robust to noise and outliers in the data.

Why each part matters (training to testing)

- Part 1: **Schema normalization.** Stabilizes names/types so engineered features map reliably to source columns; prevents header drift issues.
- Part 2: **Cleaning and log target.** Median imputation preserves central tendency; $\log(1 + \cdot)$ reduces skew and improves squared-error training stability.
- Part 3: **Feature engineering.** Encodes structural scope: floor area, concrete volume, steel, and formworks; ratios normalize for size (intensities per area/volume).
- Part 4: **9-input feature matrix.** Compact, interpretable interface that avoids target leakage and simplifies inference.
- Part 5: **Quadratic expansion + z-score.** Adds curvature and interactions while keeping features comparable in scale; improves optimization and regularization.
- Part 6: **Holdout split.** Provides an unbiased generalization estimate; all testing figures/metrics come from this split.
- Part 7: **Baseline LSBoost.** Establishes a numeric baseline (R^2 , MAE) and diagnostics (residuals, importances) to validate signal.
- Part 8: **Feature selection.** Reduces noise/variance by removing weak signals; focuses search on predictive interactions.
- Part 9: **Bayesian tuning.** Optimizes bias-variance (cycles, splits, learning rate) to improve test R^2 and MAE.
- Part 10: **Evaluation and parity.** Converts predictions back to PHP; packages assets (`mu`, `sigma`, masks, feature lists) to guarantee training-serving parity.

Inference pipeline (single row)

1. Normalize field names; compute totals (floor area, concrete volume, steel) and ratios if missing.
2. Order to final 9 features; build degree-2 features; align to training names.
3. Standardize with saved μ, σ ; apply selected feature mask; predict in log-space; invert with $\exp(\cdot) - 1$.